

PASUL 1 DIN 5 / BACKEND 5A.1

Baza de date si inregistrarea utilizatorului

Tutorial pe codul real al proiectului Netex - Java 25 - Spring Boot - PostgreSQL

Ce vei putea explica la final

Cum ajung emailul si parola dintr-o cerere HTTP intr-un rand din users, de ce parola devine hash, de unde vine ID-ul si de ce raspunsul API nu contine parola sau hashul.

Cum folosesti ghidul

Parcurge paginile in sesiuni scurte: SQL, Java, apoi verificarea practica. Tine proiectul deschis in IntelliJ si cauta fiecare fragment din PDF in fisierul real.

Dupa fiecare sesiune, povesteste traseul cererii in cuvintele tale. Nu trebuie sa memorezi adnotarile Spring.

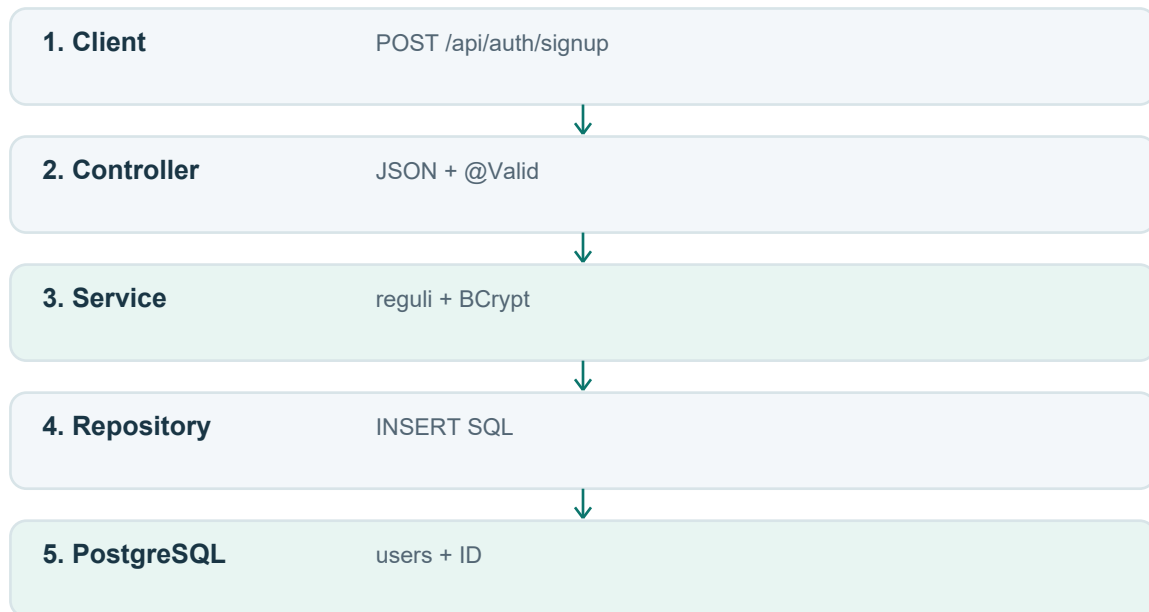
Cei cinci pasi de invatare

1. Baza de date si inregistrarea: HTTP -> validare -> hash -> SQL -> raspuns.
2. Loginul: ce citeste codul nostru din SQL si ce verifica Spring Security.
3. Sesiunea si cookie-ul: cum ramane utilizatorul logat intre cereri.
4. Permisunile: public, utilizator logat si rol ADMIN.
5. Testele: verificam fluxul si il explicam ca la interviu.

Varianta curenta: in proiectul de baza nu exista un pas separat de token CSRF. Inregistrarea se face direct cu POST /api/auth/signup. Protectia CSRF poate fi adaugata ulterior pe un branch separat.

1. Mai întâi: harta întregii operații

Imaginează-ți cererea **POST /api/auth/signup** cu un corp JSON care are **email** și **password**. Serverul trebuie să creeze contul și să întoarcă un răspuns sigur.



Pe drumul înapoi, PostgreSQL dă ID-ul nou → Repository îl transmite către Service → Service creează un **AppUser** → Controller îl transformă în **UserResponse** → clientul primește HTTP 201 și JSON.

Cele 3 locuri pe care să le ții minte

Controller = intrare/ieșire HTTP. **Service** = regulile înregistrării. **Repository** = interacțiunea cu SQL. Baza de date verifică suplimentar regulile ei, inclusiv emailul unic.

Fișierele pe care le vom citi

Fișier	Responsabilitate
V1 și V2 (SQL)	Structura tabelului users și coloana role.
SignupRequest + EmailAddress	Datele primite și normalizarea emailului.
SignupController	Ruta HTTP și răspunsul 201.
SignupService	Validare suplimentară, hash, rol USER, eroarea 409.
UserRepository	INSERT parametrizat și ID-ul întors de SQL.
AppUser + UserResponse	Datele interne versus cele expuse clientului.

2. Ce există în PostgreSQL

users conține conturile care se pot autentifica. Nu este același lucru cu un contact din agendă. Un contact va avea, ulterior, un autor în **created_by_user_id**.

V1 a creat tabelul users

backend/src/main/resources/db/migration/V1__create_users_and_contacts.sql · liniile 1-11

```
1 CREATE TABLE users (  
2     id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
3     email VARCHAR(254) NOT NULL,  
4     password_hash VARCHAR(255) NOT NULL,  
5     created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,  
6     CONSTRAINT ck_users_email_trimmed CHECK (email = btrim(email) AND email <> ''),  
7     CONSTRAINT ck_users_password_hash_not_blank CHECK (btrim(password_hash) <> '')  
8 );  
9  
10 -- The database also prevents duplicate emails that differ only in letter case.  
11 CREATE UNIQUE INDEX uk_users_email ON users (lower(email));
```

- **id BIGINT ... IDENTITY PRIMARY KEY**: PostgreSQL generează un identificator unic. Java nu îl alege.
- **email VARCHAR(254) NOT NULL**: emailul este obligatoriu și are o limită de lungime.
- **password_hash**: coloana primește hashul BCrypt, niciodată parola introdusă.
- **created_at**: baza pune automat momentul creării dacă Java nu trimite o valoare.
- **uk_users_email** pe **lower(email)**: nu permite două conturi cu același email scris cu litere diferite.

Două tipuri diferite de reguli

Java validează pentru un mesaj util către utilizator. PostgreSQL protejează datele chiar dacă alt cod încearcă să insereze un rând greșit. Indexul unic este ultima apărare și la două înregistrări simultane.

V2 a adăugat rolul fără să schimbe V1

backend/src/main/resources/db/migration/V2__add_user_role.sql · liniile 1-3

```
1 ALTER TABLE users ADD COLUMN role VARCHAR(16) NOT NULL DEFAULT 'USER';  
2  
3 ALTER TABLE users ADD CONSTRAINT ck_users_role CHECK (role IN ('USER', 'ADMIN'));
```

DEFAULT 'USER' completează și rândurile create înainte de V2 și este valoarea implicită pentru un INSERT care omite rolul. **CHECK** permite doar USER sau ADMIN. Înregistrarea publică trimite explicit USER; o proprietate **role** din JSON nu este folosită.

3. Ce intră prin HTTP

Un exemplu de corp trimis de client este `{"email": "ANA@example.com", "password": "ParolaDeTest123!"}`. Spring citește JSON-ul și construiește un obiect **SignupRequest**.

SignupRequest: forma datelor de intrare

backend/src/main/java/com/netex/addressbook/auth/signup/SignupRequest.java · liniile 8-17

```
8 public record SignupRequest(  
9     @NotBlank @Email @Size(max = 254) String email,  
10    @NotBlank @Size(min = 8, max = 72) String password) {  
11  
12    public SignupRequest {  
13        if (email != null) {  
14            email = EmailAddress.normalize(email);  
15        }  
16    }  
17 }
```

- **record** este o formă compactă de clasă pentru date; aici are două componente: email și password. Nu reprezintă un rând din SQL.
- **@NotBlank** refuză lipsa sau un text format numai din spații. **@Email** verifică forma adresei. **@Size** limitează lungimea.
- Constructorul compact normalizează emailul înainte să ruleze validarea: elimină spațiile de la capete și transformă literele în lowercase.

EmailAddress: normalizarea comună

backend/src/main/java/com/netex/addressbook/auth/EmailAddress.java · liniile 5-13

```
5 public final class EmailAddress {  
6  
7     private EmailAddress() {  
8     }  
9  
10    public static String normalize(String email) {  
11        return email.trim().toLowerCase(Locale.ROOT);  
12    }  
13 }
```

Locale.ROOT face transformarea literelor independentă de limba computerului. Metoda este folosită și la login și la inițializarea contului admin; regula stă într-un singur loc.

Nu confunda normalizarea cu validarea

Normalizarea schimbă forma unei valori acceptabile: **'ANA@EXAMPLE.COM'** devine **'ana@example.com'**.

Validarea decide dacă valoarea este permisă: **'ana'** nu este un email valid.

4. Controllerul: ușa HTTP

backend/src/main/java/com/netex/addressbook/auth/signup/SignupController.java · liniile 12-27

```
12 @RestController
13 @RequestMapping("/api/auth/signup")
14 public class SignupController {
15
16     private final SignupService signupService;
17
18     public SignupController(SignupService signupService) {
19         this.signupService = signupService;
20     }
21
22     @PostMapping
23     public ResponseEntity<UserResponse> signup(@Valid @RequestBody SignupRequest request) {
24         return ResponseEntity.status(HttpStatus.CREATED)
25             .body(UserResponse.from(signupService.signup(request)));
26     }
27 }
```

Linie / concept	Ce înseamnă
@RestController	Spring folosește această clasă pentru răspunsuri HTTP; obiectul întors devine JSON.
@RequestMapping	Prefixul rutei este /api/auth/signup.
Constructor	Spring îi oferă un SignupService. Controllerul nu îl creează cu new.
@PostMapping	Metoda răspunde la POST pe ruta clasei.
@RequestBody	Spring convertește corpul JSON în SignupRequest.
@Valid	Spring aplică adnotările de validare de pe SignupRequest.
ResponseEntity	Stabilește explicit răspunsul: status 201 Created și corp UserResponse.

Dacă **@Valid** refuză emailul sau parola, metoda din controller nu ajunge să cheme Service-ul și clientul primește HTTP 400. Dacă datele sunt valide, controllerul delegă munca la **signupService.signup(request)**.

De ce există Service-ul separat?

Controllerul nu decide rolul, nu calculează hashul și nu scrie SQL. Astfel regulile pot fi citite și testate fără să fie amestecate cu detaliile HTTP.

5. Service-ul: regulile înregistrării

backend/src/main/java/com/netex/addressbook/auth/signup/SignupService.java · liniile 13-41

```

13  @Service
14  public class SignupService {
15
16      private final UserRepository users;
17      private final PasswordEncoder passwordEncoder;
18
19      public SignupService(UserRepository users, PasswordEncoder passwordEncoder) {
20          this.users = users;
21          this.passwordEncoder = passwordEncoder;
22      }
23
24      public AppUser signup(SignupRequest request) {
25          String email = EmailAddress.normalize(request.email());
26          if (email.isEmpty() || email.length() > 254 || !email.contains("@")) {
27              throw new ResponseStatusException(HttpStatus.BAD_REQUEST, "Invalid email");
28          }
29          if (request.password().getBytes(StandardCharsets.UTF_8).length > 72) {
30              throw new ResponseStatusException(HttpStatus.BAD_REQUEST, "Password is too long");
31          }
32
33          String hash = passwordEncoder.encode(request.password());
34          try {
35              long id = users.create(email, hash, "USER");
36              return new AppUser(id, email, hash, "USER");
37          } catch (DuplicateKeyException exception) {
38              throw new ResponseStatusException(HttpStatus.CONFLICT, "Email is already registered",
39                  exception);
40          }
41      }
42  }

```

Citește metoda **signup** de sus în jos. Primește un **SignupRequest**, obține emailul normalizat, face o verificare suplimentară și transformă parola în hash. Apoi cere Repository-ului să insereze contul cu rolul fix **USER**.

Observă că **SignupRequest** a normalizat deja emailul. Apelul repetat din Service nu schimbă rezultatul; este o mică redundanță din codul actual, nu un mecanism Spring separat.

Fragment	De ce există
<code>getBytes(UTF_8).length > 72</code>	BCrypt folosește cel mult 72 de octeți; evităm acceptarea unei parole a cărei parte finală ar fi ignorată.
<code>passwordEncoder.encode(...)</code>	Generează hashul. Parola în clar nu ajunge în INSERT.
<code>users.create(..., "USER")</code>	Rolul vine de la server. Clientul nu poate alege ADMIN la signup.
<code>DuplicateKeyException</code>	Indexul unic SQL a refuzat emailul deja existent; API-ul răspunde 409 Conflict.
<code>new AppUser(...)</code>	Construiește rezultatul intern cu ID-ul primit de la PostgreSQL.

Un lucru important despre hash

Hashul nu se decriptează. La login, Spring aplică verificarea parolei introduse față de hashul salvat. Două hashuri BCrypt pentru aceeași parolă pot arăta diferit, deoarece folosesc un salt.

6. Repository-ul: operația SQL

backend/src/main/java/com/netex/addressbook/auth/UserRepository.java · liniile 7-30

```

7  @Repository
8  public class UserRepository {
9
10     private final JdbcTemplate jdbc;
11
12     public UserRepository(JdbcTemplate jdbc) {
13         this.jdbc = jdbc;
14     }
15
16     public Optional<AppUser> findByEmail(String email) {
17         return jdbc.query("""
18             SELECT id, email, password_hash, role FROM users WHERE lower(email) = lower(?)
19             """, (rs, rowNum) -> new AppUser(
20                 rs.getLong("id"), rs.getString("email"),
21                 rs.getString("password_hash"), rs.getString("role")), email)
22             .stream().findFirst();
23     }
24
25     public long create(String email, String passwordHash, String role) {
26         return jdbc.queryForObject("""
27             INSERT INTO users (email, password_hash, role) VALUES (?, ?, ?) RETURNING id
28             """, Long.class, email, passwordHash, role);
29     }
30 }
```

- **@Repository** marchează componenta care accesează datele; Spring îi furnizează **JdbcTemplate** prin constructor.
- **create** trimite către PostgreSQL un **INSERT** cu email, hash și rol. **RETURNING id** cere bazei ID-ul nou generat.
- Cele trei caractere **?** sunt locuri pentru valori. **JdbcTemplate** le trimite ca parametri, astfel emailul nu este lipit în textul SQL.
- **findByEmail** citește un cont pentru loginul viitor; **Optional** exprimă faptul că emailul poate să nu existe.

Ce este AppUser și ce este UserResponse?

backend/src/main/java/com/netex/addressbook/auth/AppUser.java · liniile 1-4

```

1  package com.netex.addressbook.auth;
2
3  public record AppUser(long id, String email, String passwordHash, String role) {
4  }
```

backend/src/main/java/com/netex/addressbook/auth/UserResponse.java · liniile 1-12

```

1  package com.netex.addressbook.auth;
2
3  public record UserResponse(long id, String email, String role) {
4
5      public static UserResponse from(AppUser user) {
6          return new UserResponse(user.id(), user.email(), user.role());
7      }
8
9      public static UserResponse from(AccountPrincipal principal) {
10         return new UserResponse(principal.id(), principal.email(), principal.role());
11     }
12 }
```

AppUser este forma internă: include **passwordHash**, deoarece loginul va avea nevoie de el. **UserResponse** este forma publică: conține numai **id**, **email**, **role**. Metoda **from(AppUser)** face conversia. De aceea hashul nu apare în JSON-ul de răspuns.

7. Rezultate posibile si limitele acestui pas

201 Created - contul a fost creat; raspunsul contine id, email si role USER.

400 Bad Request - emailul sau parola nu respecta validarea.

409 Conflict - emailul exista deja in baza de date.

Public inseamna ca utilizatorul poate trimite cererea de inregistrare fara sa fie deja logat. Varianta de baza nu cere un token CSRF pentru acest POST.

Ce nu se intampla la signup

Inregistrarea nu autentifica automat utilizatorul. Dupa raspunsul 201, GET /api/auth/me raspunde 401 pana cand utilizatorul face login. Evenimentul Kafka cerut de proiect urmeaza intr-o etapa ulterioara.

Teste relevante din proiect

In AuthApiTest, signupCreatesOnlyAUserAndNeverReturnsAHash verifica 201, rolul USER chiar daca cererea contine role ADMIN, absenta hashului din JSON, existenta hashului BCrypt in SQL si 409 pentru email duplicat.

invalidSignupsRejected verifica 400. DatabaseSchemaTest verifica migrarea V2 si constrangerea rolului.

Testele folosesc un PostgreSQL temporar pornit de Testcontainers si nu modifica baza locala netex din DBeaver. Rularea curenta a testelor trebuie refacuta cand Docker este disponibil.

8. Exercițiu practic: creeaza un cont de test

Ai nevoie de PostgreSQL pornit din Docker Compose si backendul actual pe portul 8080. Foloseste o parola de test, nu o parola personala.

A. Porneste serviciile

Din radacina repository-ului ruleaza docker compose up -d --wait postgres. Porneste backendul din IntelliJ cu profilul local sau din backend/ cu comanda din README.

B. Trimite cererea din PowerShell

Cererea POST actuala - fara pas de token

```
$email = 'student-' + (Get-Random -Minimum 10000 -Maximum 99999) + '@example.com'
$body = @{ email = $email; password = 'ParolaDeTest123!' } | ConvertTo-Json
$request = @{
    Uri = 'http://localhost:8080/api/auth/signup'
    Method = 'Post'
    ContentType = 'application/json'
    Body = $body
}
$result = Invoke-WebRequest $request
$result.StatusCode
$result.Content
```

Ar trebui sa vezi 201 si un raspuns cu ID, emailul generat si role USER. Copiaza emailul daca vrei sa folosesti contul la pasul 2.

C. Verifica vizual in DBeaver

- Deschide Schemas -> public -> Tables -> users -> View Data -> All Rows.
- Cauta valoarea din \$email; verifica id, email, role = USER si created_at.
- password_hash trebuie sa fie diferit de parola de test. Nu publica hashul.
- In flyway_schema_history verifica versiunile 1 si 2.

Daca cererea esueaza: verifica daca backendul actual ruleaza pe 8080 si daca PostgreSQL este pornit. Nu este nevoie sa apelezi /api/auth/csrf; acel endpoint a fost eliminat din varianta de baza.

9. Consolidare: poți explica singur?

Închide PDF-ul pentru 5 minute. Deschide fișierele în IntelliJ și răspunde cu voce tare, în această ordine. Dacă nu știi un răspuns, revino doar la secțiunea respectivă.

1. Unde este definit tabelul users și de ce există o migrare V2?
2. Ce primește SignupController și ce îi transmite lui SignupService?
3. Cine alege rolul USER? Ce se întâmplă dacă browserul trimite ADMIN?
4. De ce UserRepository folosește ? în INSERT și RETURNING id?
5. Unde ajunge parola în clar? Unde ajunge hashul?
6. De ce JSON-ul de răspuns nu conține password_hash?
7. De ce poate apărea 409 chiar dacă Java ar verifica în prealabil existența emailului?

Răspunsul complet de interviu, în 30 de secunde

Model de explicație

La signup, Spring primește JSON-ul în SignupController și validează SignupRequest. SignupService normalizează emailul, verifică lungimea parolei în octeți, produce un hash BCrypt și impune rolul USER. UserRepository execută un INSERT parametrizat, iar PostgreSQL generează ID-ul și garantează emailul unic. Controllerul întoarce 201 cu UserResponse, care nu include hashul. Flyway ține structura users în V1 și rolul în V2.

Ce să cauți dacă vrei să aprofundezi

- Java: **record**, constructor, **Optional** și diferența dintre obiect și rând SQL.
- Spring: **@RestController**, **@RequestBody**, **@Valid**, injecție prin constructor, **JdbcTemplate**.
- Date și securitate: **Flyway migration**, **PostgreSQL unique index**, **BCrypt PasswordEncoder**.

Nu ai nevoie să studiezi acum implementarea internă a fiecărei adnotări. Obiectivul este să poți urmări traseul cererii și să explici de ce fiecare limită există.